

Recuperação de Informação - PA2: Indexer and Query Processor

Lorenzo Carneiro Magalhães
Universidade federal de Minas Gerais
Belo Horizonte, Brasil

1 Introduction

Simulando os conceitos teóricos vistos em sala de aula, este trabalho consistiu na construção de um indexador e de um processador de consultas (queries). O principal desafio, no entanto, está em realizar todo esse processamento de forma eficiente, tanto em termos de uso de memória quanto de velocidade de execução. Este desafio é particularmente relevante, pois, no contexto de Recuperação de Informação e de motores de busca, o volume de dados costuma ser significativamente maior do que a capacidade de memória disponível na máquina. Isso exige o desenvolvimento de soluções computacionais inteligentes e cuidadosas, capazes de realizar operações como gerenciamento de memória, processamento incremental e técnicas de intercalação externa, de modo a garantir que o sistema funcione corretamente mesmo sob restrições de recursos.

2 Indexador

A estratégia adotada para o desenvolvimento do indexador teve como foco principal garantir um uso eficiente da memória, ao mesmo tempo em que preservava a escalabilidade e a velocidade de processamento. Para isso, a arquitetura foi estruturada de forma modular e orientada a objetos. A implementação contou com 3 componentes principais.

O primeiro componente é a classe MemHandler, que ficou responsável por todo o gerenciamento da memória e leitura durante o processo de indexação. Essa classe monitora constantemente o uso de memória, lê o corpus e controla quando os índices parciais devem ser descarregados para disco, garantindo que o consumo nunca ultrapasse o limite previamente estabelecido.

O segundo componente consiste no módulo responsável pela própria indexação dos documentos. Esse módulo realiza as etapas de pré-processamento, como remoção de stopwords e aplicação de stemming, e construção das três principais estruturas do sistema: o índice invertido, o índice de documentos e o léxico de termos.

Por fim, a classe principal, chamada Indexer, exerce o papel de orquestradora do processo. Ela gerencia as múltiplas threads de execução, coordena o fluxo de documentos e controla a execução paralela do pipeline de indexação.

2.1 Ideia geral

O funcionamento do algoritmo segue uma lógica eficiente e incremental. A leitura do corpus é feita por meio de uma função geradora, que retorna um documento por vez. Esse design foi essencial para permitir que as threads solicitassem documentos de forma independente, possibilitando um fluxo contínuo de dados sem sobrecarregar a memória do sistema. Cada thread, ao receber um documento, aplica o pré-processamento e atualiza as estruturas locais mantidas em memória: os dicionários que representam o índice invertido, o índice de documentos e o léxico de termos. Esse ciclo se repete continuamente até que o MemHandler identifique que o uso de memória está se aproximando do limite configurado.

Quando isso ocorre, o MemHandler entra em ação e executa o descarregamento dos índices parciais para arquivos temporários no disco, liberando assim espaço na memória para que o processamento continue normalmente. Essa estratégia permite que o indexador opere sobre conjuntos de dados cujo volume é muito superior à memória disponível na máquina.

Ao término do processamento de todo o corpus, os arquivos de índices parciais precisam ser consolidados em um índice único e consistente. Para isso, foi utilizada a técnica de External Merge Sort, que é um algoritmo utilizado para ordenar grandes volumes de dados que não cabem na memória principal. O algoritmo consiste em carregar os arquivos parciais contendo os termos lexicograficamente ordenados e mesclar as listas de ocorrências associadas a cada termo provenientes dos diferentes arquivos. Esse processo garante que o índice invertido final esteja devidamente ordenado, o que é essencial para permitir buscas rápidas e eficientes no momento do processamento das consultas e também facilidade na própria junção dos termos.

O índice do documento pode ser salvo incrementalmente e não há necessidade da construção de vários arquivos, já que não há relação entre documentos. O léxico pode ser construído tanto durante o processo do External Merge Sort quanto no processo de indexação propriamente dito. Na minha implementação, por facilidade optei por fazer durante a ordenação externa.

2.2 Pré-processamento

Antes da execução da etapa de indexação do corpus, foi realizado um processo de pré-processamento dos dados com o objetivo de preparar os textos para uma representação mais eficiente e compacta, facilitando, assim, a posterior recuperação da informação. Essa etapa mostrou-se fundamental para assegurar uma execução mais rápida e eficaz dos algoritmos de busca adotados.

Conforme especificado no enunciado do trabalho, o pré-processamento incluiu a remoção de stopwords da língua inglesa, idioma predominante nos textos do corpus, utilizando a biblioteca NLTK. Além disso, foram eliminados caracteres especiais, sinais de pontuação e outros símbolos que não contribuíam de forma relevante para a representação dos documentos. Ao final desse processo, mantiveram-se apenas os caracteres alfanuméricos. Em seguida, aplicou-se o algoritmo PorterStemmer, também disponibilizado pela NLTK, com o intuito de realizar a normalização morfológica dos termos e, assim, aumentar a revocação nas consultas. A tokenização foi implementada de maneira simples, dividindo o texto em palavras com base nos espaços em branco.

Cabe destacar uma observação importante em relação à preservação de caracteres numéricos. Verificou-se que a presença de certos números pode interferir negativamente nos resultados das consultas conjuntivas, desviando o foco semântico e reduzindo a revocação. Essa conclusão foi reforçada por uma análise empírica dos resultados: identificaram-se ocorrências de “palavras” compostas

exclusivamente por dígitos numéricos, como “0029052009”, que provavelmente representam datas ou identificadores específicos, mas que, no contexto da indexação tradicional, não apresentam relevância significativa.

2.3 Gerenciamento de memória

Durante o desenvolvimento, o principal desafio encontrado esteve diretamente relacionado ao gerenciamento de memória, mais especificamente na dificuldade em estimar com precisão a quantidade de memória realmente utilizada pela aplicação. Embora a função `rss` recomendada na especificação tenha sido empregada para monitoramento, na prática ela não refletia exatamente a memória efetivamente alocada pelos objetos em Python, o que tornava o controle mais complexo do que o esperado. Pelo o que li na internet, essa função reflete a memória que o sistema operacional alocou para o processo em questão, o que não necessariamente mostra o uso atual e sim um limite superior do uso.

Para superar essa dificuldade, foi desenvolvido um algoritmo adaptativo que ajusta dinamicamente o número de documentos processados antes de realizar a operação de descarregamento/salvamento dos índices para disco. A estratégia adotada inicia com um valor conservador para o número de linhas processadas, evitando assim que a memória seja saturada logo nas primeiras iterações. A cada novo ciclo de salvamento, o sistema verifica se o uso atual de memória está abaixo de 90% do limite estipulado. Caso haja folga, o número de linhas processadas (*batch size*) é incrementado, permitindo maior eficiência por reduzir a frequência de operações de escrita em disco. Por outro lado, se o uso de memória se aproxima do limite, o sistema automaticamente ativa um modo de proteção, reduzindo o tamanho do batch e garantindo que não se ultrapasse 90% da memória disponível.

Além disso, foi implementado um mecanismo adicional de segurança, chamado de *safeguard*, que introduz uma margem extra para assegurar que, mesmo em situações de picos momentâneos de alocação, o sistema não atinja o limite máximo de memória, prevenindo assim exceder a memória especificada.

Essa estratégia de controle dinâmico da memória se mostrou bem robusta, permitindo encontrar um equilíbrio adequado entre desempenho e segurança: na medida em que reduz a quantidade de operações, garante que o uso da memória permaneça sempre dentro dos limites seguros. A combinação desse controle com a utilização da intercalação externa garantiu que fosse possível processar um corpus de tamanho expressivo sem comprometer a estabilidade nem a eficiência do sistema.

O problema dessa abordagem é que o valor inicial pode ser conservador demais a ponto de que a convergência para o valor ótimo demore uma quantidade muito grande de ciclos de salvamento. O aumento e a redução do número de linhas processadas por salvamento foram mantidos predefinidos por fins de simplicidade.

3 Processador

Com o índice invertido construído e persistido em disco, a próxima etapa do sistema consiste na implementação de um processador de consultas capaz de realizar a recuperação dos documentos mais relevantes para cada entrada textual. O objetivo desta etapa é garantir

uma busca eficiente, precisa e baseada em modelos tradicionais da área, como o TF-IDF e o BM25.

Ao iniciar sua execução, o processador carrega os arquivos com os índices previamente construídos: o índice invertido, o índice de documentos (com o tamanho de cada documento) e o léxico (com a frequência de documentos por termo). As consultas são então lidas de um arquivo externo e processadas de forma sequencial, passando por um pipeline de normalização textual semelhante ao aplicado nos documentos do corpus. Esse alinhamento entre os dois estágios é fundamental para garantir a consistência semântica e sintática entre o que foi indexado e o que está sendo consultado.

Uma vez pré-processada, cada consulta é submetida a um mecanismo de seleção baseado em um operador do tipo DAAT. A versão adotada neste trabalho é conjuntiva, conforme a especificação, o que significa que apenas os documentos que contêm todos os termos presentes na consulta são considerados candidatos à recuperação. Essa decisão está alinhada com o objetivo de promover precisão nos resultados, mesmo ao custo de certa perda de revocação. A estratégia, no entanto, não se mostrou muito tenaz, em muitos experimentos consultas simples não trouxeram resultados.

4 Modelos de Ranqueamento

Após a seleção dos documentos candidatos, o sistema aplica um modelo de ranqueamento para determinar a relevância de cada documento em relação à consulta. Ambos os modelos TF-IDF e BM25 foram implementados.

O modelo TF-IDF utiliza uma abordagem clássica e simples em que a frequência do termo no documento é normalizada pelo seu tamanho, e ponderada pela frequência inversa do termo no conjunto de documentos candidatos. A métrica resultante busca destacar documentos onde os termos da consulta ocorrem com intensidade, mas sem serem triviais ou redundantes no restante do corpus.

O modelo BM25, por sua vez, incorpora uma estratégia de normalização mais refinada, levando em conta a saturação da frequência de termos e o comprimento médio dos documentos candidatos. Os parâmetros k e b foram fixados conforme valores tradicionalmente utilizados na literatura, garantindo estabilidade nos resultados. A frequência de documentos por termo também é trabalhada para evitar distorções em casos extremos.

5 Resultados

No geral, os resultados obtidos foram dentro do esperado: apesar da indexação demorar bastante, a indexação foi eficiente para produzir bons resultados de maneira eficiente e não ultrapassando a memória especificada. O processador também produziu resultados bons e de maneira rápida, de maneira a alinhar com o objetivo da ferramenta, que é fornecer boas indicações de documentos ao usuário final de maneira rápida.

Um estudo sobre a quantidade ideal de threads no indexador foi feita e o valor ideal foi um pouco diferente do esperado. A quantidade de threads que alcançou o melhor resultado foi 1 thread apenas. Qualquer número a mais de threads resultava em uma demora no processamento. Para 1 milhão de documentos, 8 threads demoram mais de 250 segundos a mais do que 1 thread, o que é um resultado expressivo.

Esse resultado pode ser explicado pela quantidade de interações bloqueantes, como salvamentos no disco e acessos muito frequentes às estruturas. Na tabela 1 pode-se perceber essa tendência do aumento de tempo de execução à medida que o número de threads aumenta.

Número de threads	Tempo (s)
1	420.69
2	434.39
4	625.51
8	694.11

Table 1: Tempo de execução com relação ao número de threads.

A partir da análise dos resultados obtidos com o processamento das consultas, foi possível verificar que o sistema apresentou um desempenho satisfatório em termos de relevância e rapidez. Para esclarecimento da eficácia do método, a próxima seção mostrará os retornos de uma consulta arbitrária.

6 Estudo de Caso: consulta "newton law"

Para ilustrar o funcionamento do sistema de recuperação de informação, foi realizada a consulta textual "newton law". A seguir, serão apresentados os três documentos mais relevantes retornados, segundo o modelo de ranqueamento BM25, seguidos por alguns exemplos com pontuação menor.

Top 3 resultados

- **Newton (unit)**

"The newton (symbol: N) is the International System of Units (SI) derived unit of force. It is named after Isaac Newton in recognition of his work on classical mechanics, specifically Newton's second law of motion."

Keywords: SI derived units

- **Saverland v Newton**

"Saverland v. Newton (1837) is a notorious court case in which a British gentleman, Thomas Saverland, brought an action against Miss Caroline Newton, who had bitten the left half of his nose off after he attempted to steal a kiss at a party."

Keywords: 1837 in British law, English tort case law, Kissing

- **Newton's law of cooling**

"Newton's law of cooling states that the rate of heat loss of a body is proportional to the difference in temperatures between the body and its surroundings while under the effects of a breeze."

Keywords: Heat conduction, Heat transfer, History of physics

Outros documentos menos relevantes

- **Newton Centre, Massachusetts**

"Newton Centre is a village of Newton, Massachusetts. The main commercial center is a triangular area surrounding

Beacon Street, Centre Street, and Langley Road."

Keywords: Villages in Massachusetts

- **Eben Newton**

"Eben Newton (1795–1885) was a U.S. Representative from Ohio. He studied law and served as senator in Ohio during the 19th century."

Keywords: Ohio Whigs, U.S. Representatives from Ohio

- **The Newtones**

"The Newtones is a coed student-run a cappella group from Newton South High School in Massachusetts. They sing a variety of genres including pop and folk."

Keywords: Musical groups from Massachusetts, A cappella

Esses resultados demonstram a capacidade do sistema em recuperar documentos diretamente relacionados à consulta mesmo em meio a diversas ambiguidades advindas do nome 'newton' e do termo 'law'. Isso mostra que o ranqueamento é eficaz em priorizar documentos contextualmente relevantes à consulta original.